

InsurMinds — Desafio 5

Ferramenta Inteligente para Comunicação Proativa com o Segurado

Relatório Técnico

Grupo InsurTech Minds

Ednardo Pinheiro Peixoto — Representante e Engenheiro de IA/IoT/Tech Líder

Glenda dos Santos Tavares — Atuária

Sueli Da Hora Moreira — Analytics Engineer

Rafael Torres Lattaro Soares — Especialista em Seguros

Instituto de Inteligência Artificial Aplicada — I2A2

1. Objetivo

Este documento descreve a solução desenvolvida pelo grupo InsurTech Minds para o Desafio 5 do InsurMinds: um protótipo funcional (MVP) capaz de monitorar condições meteorológicas em tempo real e gerar comunicações preventivas e personalizadas para segurados, antes que um sinistro ocorra — transformando o modelo reativo de relacionamento das seguradoras em uma abordagem proativa.

2. Contexto

Grande parte das interações entre seguradoras e clientes acontece apenas após a ocorrência de um sinistro. A solução aqui descrita monitora eventos climáticos externos (chuva intensa, vento forte, tempestades com risco de granizo) e envia orientações preventivas personalizadas aos segurados potencialmente afetados, de acordo com o tipo de seguro contratado e a cidade onde residem.

3. Arquitetura da solução (multiagente)

A solução foi estruturada como um fluxo de 5 agentes especializados, conforme sugerido pelo próprio edital do desafio, cada um implementado como um módulo Python independente em `src/`, orquestrados pela interface Streamlit (`app.py`):

- **Agente 1 — ColetorClimaticoAgent** (`weather_agent.py`): consulta a API pública OpenWeatherMap (Current Weather Data) para cada cidade da base de segurados, coletando temperatura, condição do tempo, velocidade do vento e volume de chuva.
- **Agente 2 — AnalisadorEventosAgent** (`event_agent.py`): aplica limiares configuráveis para identificar eventos relevantes: chuva intensa (≥ 10 mm/h), vento forte (≥ 12 m/s) e tempestade/risco de granizo (código meteorológico do grupo "Thunderstorm" da OpenWeatherMap, usado como proxy, já que o plano gratuito não informa granizo diretamente).
- **Agente 3 — RegrasNegocioAgent** (`rules_agent.py`): cruza os eventos identificados com a base de segurados, aplicando regras de negócio que definem quais tipos de seguro devem ser notificados para cada tipo de evento.
- **Agente 4 — GeradorMensagemAgent** (`message_agent.py`): usa um modelo de linguagem (Google Gemini) para redigir uma mensagem curta, personalizada e preventiva para cada segurado afetado, com tom profissional e recomendações concretas.
- **Agente 5 — SimuladorEnvioAgent** (`notify_agent.py`): registra o envio simulado de cada notificação (nome, canal, evento, mensagem, horário), sem nenhuma integração real de SMS/e-mail, conforme os requisitos do desafio.

Fluxo de funcionamento: coleta de dados climáticos → identificação de eventos → aplicação de regras de negócio → geração de mensagem personalizada por IA → simulação do envio, com cada etapa exibida na interface Streamlit em tempo real.

4. Tecnologias utilizadas

- Python 3.10+
- Streamlit — interface única, exibindo o fluxo completo em tempo real

- requests — consumo da API pública OpenWeatherMap
- LangChain + Google Gemini (gemini-flash-lite-latest, camada gratuita) — geração das mensagens personalizadas; também suporta Ollama (local/gratuito), Claude e GPT (pagos), configurável via variável de ambiente
- pandas — manipulação da base de segurados

5. Fonte de dados meteorológicos

Foi utilizada a API pública **OpenWeatherMap** (Current Weather Data), com camada gratuita, consultada em tempo real para cada cidade presente na base de segurados. Documentação: <https://openweathermap.org/current>

6. Base de segurados (fictícia)

Como o desafio não disponibiliza uma base real de clientes, foi criada uma base fictícia (data/segurados.csv) com 20 segurados distribuídos em 10 cidades brasileiras — incluindo regiões costeiras (Fortaleza, Recife, Salvador, Rio de Janeiro, Santos, Natal, Florianópolis) e do interior/sudeste-sul (São Paulo, Belo Horizonte, Porto Alegre) — cobrindo dois tipos de seguro: Residencial e Automóvel, cada segurado com um canal de contato preferido (SMS ou E-mail).

7. Regras de negócio implementadas

- **Chuva intensa** → notifica segurados com seguro Residencial
- **Tempestade (risco de granizo)** → notifica segurados com seguro Automóvel e Residencial
- **Vento forte** → notifica segurados com seguro Residencial (ênfase em regiões costeiras)

8. Execução real e evidências

A aplicação foi executada de ponta a ponta, em ambiente real, consultando a API OpenWeatherMap para as 10 cidades da base de segurados no dia 07/09/2026. O sistema identificou automaticamente um evento de **chuva intensa em São Paulo/SP** (27,2 mm/h), aplicou corretamente a regra de negócio (apenas o segurado com seguro Residencial na cidade foi selecionado) e gerou, via IA generativa, a seguinte mensagem personalizada:

Segurado: Patrícia Gomes (São Paulo/SP) · Canal: E-mail · Evento: chuva_intensa (severidade alta)

"Olá, Patrícia Gomes. Identificamos uma chuva intensa em São Paulo, com volume de 27,2 mm/h, e queremos ajudar a proteger seu patrimônio. Para evitar transtornos, recomendamos que verifique a limpeza das calhas e ralos, e afaste os eletrônicos das tomadas baixas. Conte com a nossa segurança para zelar pelo seu lar."

Status registrado pelo Agente 5: *"simulado (nenhum envio real foi realizado)"*, com timestamp da execução — confirmando que nenhuma comunicação real foi disparada, conforme exigido pelo edital. As demais 9 cidades monitoradas na mesma execução não apresentaram eventos relevantes no momento da consulta (dados brutos de todas as cidades — temperatura, condição, vento e chuva — foram exibidos na interface para fins de auditoria).

Evidências adicionais desta execução (capturas de tela da interface e exportação dos dados climáticos brutos consultados) estão disponíveis na pasta DOCS/ do repositório do projeto.

9. Requisitos mínimos atendidos

- Consumir dados de pelo menos uma API pública de informações meteorológicas (OpenWeatherMap). ✓
- Identificar automaticamente eventos climáticos relevantes. ✓
- Aplicar regras de decisão para determinar quais segurados devem receber notificações. ✓
- Gerar mensagens personalizadas utilizando Inteligência Artificial (Google Gemini). ✓
- Simular o envio das notificações, sem integração real. ✓
- Demonstrar o fluxo completo, da obtenção dos dados até a geração da comunicação. ✓

10. Limitações conhecidas e possibilidades de evolução

- O risco de granizo é inferido via proxy (condição "Thunderstorm" da OpenWeatherMap), já que o plano gratuito da API não reporta ocorrência de granizo diretamente — uma evolução seria integrar uma fonte de dados especializada (ex.: radar meteorológico) para maior precisão.
- A base de segurados é fictícia e estática (CSV); em um cenário real, seria substituída por uma integração com o sistema de cadastro da seguradora.
- Os eventos dependem do clima real no momento da execução — para fins de demonstração e testes, os limiares de chuva/vento podem ser ajustados via variáveis de ambiente.
- Evolução natural: histórico de notificações em banco de dados, agendamento automático (cron) para monitoramento contínuo, e integração real com canais de envio (SMS/e-mail) via um gateway.

11. Entregáveis desta submissão

- Este relatório técnico em PDF.
- Repositório público no GitHub, com código-fonte completo, README com instruções de instalação/execução e licença MIT:
<https://github.com/EDNARDOPPEIXOTO99/insurminds-desafio5-comunicacao-proativa>

Fim do relatório.